

DOCUMENT 03

Text-shaped cleaning algorithm

Mask construction, padding, context and an exact write-boundary contract

1. Two modes, one invariant for the new mode

Confirmed requirement: preserve the existing behavior and add a text-shaped option. The following algorithm is a proposed implementation.

Mode	Behavior
legacy	Execute the existing geometry and routing path unchanged. Do not reinterpret old records as filled rectangles.
text_shape	Build a lettering-derived footprint, add explicit padding, preview it, and constrain every committed edit to it.

The mode controls geometry, not the detector or engine. A CTD-only run can use text_shape. A CTD + COO run can use legacy. Both combinations should remain testable.

Separate the five geometries

Detection bounds: a rectangle/polygon used to localize a candidate, merge duplicates and request a crop.

Refinement context: surrounding original pixels inspected to distinguish lettering from background. It may exceed the candidate bounds.

Base lettering mask B: the best current estimate of text pixels, including text-owned outlines/shadows where established. It is immutable for a given revision.

Allowed write support W: B after the selected grouping and padding operations. This is the visible “box” the user asked to change into a shape.

Model context / hole: the crop and missing-pixel input used by the engine. They are implementation inputs, never permission to overwrite a larger region.

Required invariant

```
For every source pixel p outside W:  
  committed_output[p] = input_composite_below_this_layer[p]  
  
For the complete visible page:  
  changed_pixels_from_source is a subset of  
  the union of visible applied layer masks.
```

This is a sample-value guarantee, not a promise of identical compressed-file bytes. Metadata preservation is an independent export requirement. The current compositor already follows a mask-bounded design. [\[S11\]](#)

2. Build a trustworthy base mask

Proposed processing sequence

A1 - Discover. Run the selected detectors. Retain page-coordinate candidates and detector provenance; no text transcription is requested.

A2 - Establish context. Expand a local crop for refinement. For long strips, respect the verified source/decode window. Do not make the detection rectangle a hard glyph boundary when an outline extends beyond it.

A3 - Collect segmentation evidence. Use CTD's probability map. For COO-only or weak candidates, rerun segmentation on an enlarged local crop without requiring a second CTD box. Preserve confidence maps before thresholding where practical.

A4 - Refine against the source image. Evaluate foreground candidates using local color, polarity and component evidence guided by the text prediction. Cover dark text, white text and colored/outlined text. CTD's upstream implementation supplies a useful reference for this type of refinement. [S21]

A5 - Protect non-text. Flag components that extend into faces, speed lines, borders or broad same-colored artwork. A strong edge is evidence, not a universal stopping rule: letters themselves have strong edges. Avoid keeping an entire connected dark component just because one part intersects a text seed.

A6 - Keep small real components. Do not delete punctuation, detached Korean character parts, dots, furigana or isolated SFX fragments with a single global area threshold. Use scale-aware confidence and context.

A7 - Save B. Store a source-coordinate base mask and its provenance. If evidence is inadequate, return "mask needs correction," not a full rectangle.

Outlines, shadows and glows

The target is the lettering as rendered, not only its dark center. Include a text-owned outline or shadow in B when supported by the image. A broad translucent glow is ambiguous; present a preview and correction control rather than claiming perfect automatic separation.

Do not derive B from the cyan in the supplied references. Those overlays explain intent; they are not clean source pixels or trusted labels.

Failure ladder

Refined CTD -> optional second segmenter -> user mask correction / review. A fallback may reduce automation; it must not silently expand the edit to the full detected region.

3. Footprint, padding and topology

Default proposal: rounded expansion of the base

For source-coordinate pixel centers and a requested radius r :

$$W(r) = \{p \text{ in the valid source domain} : \text{distance}(p, B) \leq r\}$$

Use a Euclidean distance transform or a consistently defined circular structuring element. Record the convention, including how integer boundary pixels are treated. Dilation expands the foreground according to the structuring element; a square and a circle do not produce the same footprint. [S31]

Zero means zero. At $r = 0$, W equals B unless an explicitly selected grouping operation changes it. Avoid inherited minimum growth in `text_shape` mode.

Source pixels, not display pixels. Zooming, device-pixel ratio or detector resize must not change the applied padding. If a future relative/stroke-width option is added, display and persist the resolved source-pixel radius.

Rebuild from the base. Padding $2 \rightarrow 5 \rightarrow 2$ must reproduce the first 2-pixel mask exactly. Do not apply dilation to a previously dilated result or use an old applied mask as the new base.

Optional compact-group shape

The reference may be satisfied by disconnected padded glyphs or a gently connected text footprint. Start with the former. A small, previewed closing/gap-bridging parameter may connect nearby letters within the same text group; it must not connect distant SFX fragments or span a large artwork gap.

Do not use a bounding rectangle, minimum-area rectangle or global convex hull as the default erase shape. Preserve interior holes unless the user or a reviewed mask rule intentionally includes them. Closed character counters may contain real background art.

Parameter proposal, not settled defaults

Setting	Trial values	Rule
Edge padding	0, 1, 2, 3, 5 source px	2 px is a candidate default only.
Group-gap closing	Off initially	Separate from padding; show its extra area.
Feather	0-1 source px initially	Must remain inside W and not leave original glyphs visible.
Context expansion	Engine/refiner-dependent	Reading more context never authorizes more editing.

Do not add CTD's reference dilation, legacy first-step growth and a new user radius on top of each other. One mask plan must own the resolved geometry.

4. Engine and compositing contract

Build once; all engines consume the same plan

```
MaskPlan:
  base_mask_B
  write_support_W
  optional_blend_alpha_A
  refinement_context
  engine_context
  model_hole_H
  source_revision
  base_hash, support_hash, plan_revision
```

W is the maximum permitted footprint. A is zero outside W. At the confirmed text pixels, A must be one unless the user deliberately chose a partial-strength edit.

Recommended first version: $H = W$ and hard compositing ($A = 1$ in W). Add inward feathering only after measuring halos and residual text. A larger internal model hole can be a future experiment, but its output still cannot be committed outside W.

Inward feathering without leakage

A possible alpha rule is:

```
A(p) = 0           when p is outside W
A(p) = 1           when p is in B
A(p) = min(1, d_inside_W(p) / f)  otherwise, when f > 0
A(p) = 1           otherwise, when f = 0
```

Define pixel-center distances consistently. Test narrow strokes and thin padding. A feather wider than the available margin must not reduce opacity on text itself. Do not apply an unbounded blur and let nonzero alpha leak outside W.

Apply through a bounded patch

The engine may generate a rectangular output crop. Start the patch buffer as a copy of the source composite beneath the layer, blend only where A is nonzero, and store W as the authoritative applied mask. The final compositor must enforce W again.

For overlapping layers, rebuild a re-run against the intended lower-layer composite, excluding the layer being replaced. Keep the immutable source mask reference separate from the inpainting context reference. A padding reduction must restore previously covered pixels rather than bake them into a new source.

Existing isolation margins need explicit handling

The current model path has separate ink and isolation geometry. [S08] Simply setting Fitted.mask to a tight shape or passing manual=true is insufficient to prove that later stages will not expand it.

Add an explicit new-mode contract to model geometry, denoise, fill and any enabled per-region engine. When an engine cannot honor that contract, decline it visibly in this mode; never fall back to a wider hidden edit.

5. Preview identity, errors and implementation guidance

One authoritative mask revision

The backend prepares W and returns a mask reference plus source revision, mode, parameters and support hash. The canvas draws that mask. Apply requests identify the same revision. Reject a stale preview after a source, mask, padding, engine-geometry or grouping change.

Contours are useful for outlines, but a simplified contour is not the authority for writes. Use the raster footprint for fill/coverage and the contour only for presentation. A storage bbox or selection handle may remain rectangular without affecting W.

Validation before an engine runs

Check dimensions, crop transforms, finite parameters, clipping domain, nonempty support and source identity. Confirm B is contained in W. If protected artwork intersects B, report a conflict rather than silently shaving off lettering or erasing the protected mark.

Avoid treating detector confidence as mask correctness. Useful review diagnostics include mask area versus candidate area, how much refinement added, border/foreground ambiguity, unfilled outline evidence, and disagreement between mask sources. These are diagnostic features until calibrated.

Pseudocode contract

```
prepare(candidate, source, options):
    evidence = segment_with_context(candidate, source)
    B = refine_text_pixels(evidence, source, options.refinement)
    if B is empty or uncertain_beyond_policy:
        return NeedsCorrection(reason)
    B = apply_explicit_user_mask_corrections(B)
    W = derive_footprint_from_base(B, options)
    A = bounded_alpha(B, W, options.feather)
    assert B subset_of W
    assert support(A) subset_of W
    return immutable_plan(B, W, A, source.revision, options)

apply(plan, source_below, engine):
    require_matching_source_and_plan_revision(plan)
    generated_crop = engine.render(plan.context, plan.hole)
    patch = source_below.copy_crop(plan.W.bounds)
    blend_only_inside(patch, generated_crop, plan.A)
    commit_patch(patch, applied_mask=plan.W)
```

This is design pseudocode, not a compiled Rust patch. An implementation must integrate cancellation, color/depth handling, error propagation, source-window constraints and existing persistence APIs.

Completion criterion: preview, patch mask and exported applied mask refer to the same W; pixels outside the visible support remain unchanged for every engine supported by the new mode.

Source IDs link directly to primary references. Full evidence ledger: Document 08.